Unidad 1 — Fundamentos de la Web

Índice

1. Internet: concepto, historia e infraestructura
2. La Web: origen y evolución
3. Páginas web, sitios web y tipos de sitios
4. Arquitectura cliente-servidor
5. URLs y DNS
6. Protocolo HTTP/HTTPS
7. Herramientas de desarrollo del navegador
8. Introducción a HTML5
9. Estructura de un documento HTML5
10. Etiquetas básicas de HTML
11. Elementos semánticos
12. Validación según estándares del W3C

---

1. Internet: concepto, historia e infraestructura

¿Qué es Internet?

Internet es una red global de redes de computadoras interconectadas que se comunican entre sí utilizando un conjunto estandarizado de protocolos. No es una entidad única ni centralizada, sino una infraestructura distribuida que conecta millones de dispositivos en todo el mundo.

Es importante entender que Internet no es sinónimo de la Web. Internet es la infraestructura de red subyacente; la Web es uno de los muchos servicios que funcionan sobre esa infraestructura, junto con el correo electrónico, la transferencia de archivos (FTP), la mensajería instantánea, las videollamadas y muchos otros.

Breve historia de Internet

Internet tiene sus orígenes en proyectos de investigación militar y académica en los Estados Unidos:

1969 — ARPANET: La Agencia de Proyectos de Investigación Avanzados del Departamento de Defensa de los Estados Unidos (ARPA, luego DARPA) creó ARPANET, la primera red de computadoras que utilizaba conmutación de paquetes. El 29 de octubre de 1969 se envió el primer mensaje entre la Universidad de California en Los Ángeles (UCLA) y el Instituto de Investigación de Stanford (SRI). El mensaje era "LOGIN", pero el sistema se cayó después de las letras "LO".

1974 — TCP/IP: Vinton Cerf y Robert Kahn publicaron el diseño del protocolo TCP (Transmission Control Protocol), que junto con IP (Internet Protocol) se convertiría en el estándar de comunicación de Internet. TCP/IP fue adoptado oficialmente por ARPANET en 1983.

1983 — DNS: Se creó el Sistema de Nombres de Dominio (DNS), que permite traducir nombres legibles por humanos (como ejemplo.com) en direcciones IP numéricas.

1989 — La World Wide Web: Tim Berners-Lee, investigador del CERN (Organización Europea para la Investigación Nuclear) en Suiza, propuso un sistema de hipertexto distribuido que se convertiría en la World Wide Web.

1991 — Internet se abre al público: La red dejó de ser exclusivamente académica y militar, permitiendo el acceso comercial y público.

1993 — Primer navegador gráfico: La aparición de navegadores con interfaz gráfica hizo que la Web fuera accesible para usuarios no técnicos, impulsando su adopción masiva.

2000 en adelante: La banda ancha, los dispositivos móviles, las redes sociales, la computación en la nube y la inteligencia artificial transformaron Internet en la infraestructura esencial que conocemos hoy.

Infraestructura básica de Internet

Internet funciona gracias a una combinación de componentes físicos y lógicos:

Componentes físicos:

- **Cables submarinos:** Más del 95% del tráfico intercontinental de Internet viaja a través de cables de fibra óptica tendidos en el fondo de los océanos.
- **Centros de datos (data centers):** Grandes instalaciones que alojan servidores y equipos de red.
- **Routers y switches:** Dispositivos que dirigen el tráfico de datos entre redes.
- **Puntos de intercambio de Internet (IXP):** Ubicaciones físicas donde diferentes redes se interconectan para intercambiar tráfico.
- **Última milla:** La conexión final entre el proveedor de Internet y el usuario (fibra óptica, cable coaxial, DSL, redes móviles, satélite).

Componentes lógicos:

- **Direcciones IP:** Identificadores numéricos únicos asignados a cada dispositivo conectado a Internet. Existen dos versiones: IPv4 (ejemplo: 192.168.1.1) e IPv6 (ejemplo: 2001:0db8:85a3:0000:0000:8a2e:0370:7334).
- **Protocolos:** Conjuntos de reglas que definen cómo se comunican los dispositivos.
- **Proveedores de servicios de Internet (ISP):** Empresas que proporcionan acceso a Internet a usuarios y organizaciones.

Protocolos de red: el modelo TCP/IP

Un protocolo de red es un conjunto de reglas y convenciones que definen cómo se transmiten los datos entre dispositivos. El modelo TCP/IP organiza estos protocolos en cuatro capas:

Capa de acceso a la red (enlace): Se encarga de la transmisión física de los datos a través del medio (cable, WiFi, fibra óptica). Incluye protocolos como Ethernet y WiFi.

Capa de Internet (red): Se encarga del direccionamiento y el enrutamiento de los paquetes de datos entre redes. El protocolo principal es IP (Internet Protocol), que asigna direcciones a cada dispositivo y determina la ruta que seguirán los datos.

Capa de transporte: Garantiza que los datos lleguen completos y en orden. Los dos protocolos principales son:

- **TCP (Transmission Control Protocol):** Orientado a la conexión. Garantiza la entrega ordenada y completa de los datos. Se usa para páginas web, correo electrónico y transferencia de archivos.
- **UDP (User Datagram Protocol):** No orientado a la conexión. Más rápido pero sin garantía de entrega. Se usa para streaming de video, videollamadas y juegos en línea donde la velocidad es más importante que la perfección.

Capa de aplicación: Contiene los protocolos que usan las aplicaciones para comunicarse. Aquí se encuentran HTTP (para la Web), SMTP (para correo electrónico), FTP (para transferencia de archivos), DNS (para resolución de nombres) y muchos otros.

Cuando visitamos una página web, todas estas capas trabajan en conjunto: la capa de aplicación genera la solicitud HTTP, la capa de transporte (TCP) la divide en segmentos y garantiza su entrega, la capa de Internet (IP) la enruta hacia el servidor correcto, y la capa de acceso a la red la transmite físicamente a través del medio disponible.

2. La Web: origen y evolución

¿Qué es la Web?

La World Wide Web (WWW o simplemente "la Web") es un sistema de documentos de hipertexto interconectados que se acceden a través de Internet. Fue inventada por Tim Berners-Lee en 1989 y se basa en tres tecnologías fundamentales:

- **HTML (HyperText Markup Language):** El lenguaje de marcado para crear documentos web.
- **HTTP (HyperText Transfer Protocol):** El protocolo de comunicación para transferir esos documentos.
- **URL (Uniform Resource Locator):** El sistema de direcciones para localizar cada recurso en la Web.

Diferencia entre Internet y la Web

Esta es una confusión muy común que es importante aclarar:

- **Internet** es la infraestructura de red global: los cables, routers, protocolos y conexiones que permiten que los dispositivos se comuniquen entre sí.
- **La Web** es un servicio que funciona sobre Internet: un sistema de documentos enlazados que se acceden a través de navegadores.

Una analogía útil: Internet es como la red de carreteras de un país, y la Web es como el sistema de correo postal que usa esas carreteras. El correo necesita las carreteras para funcionar, pero las carreteras también sirven para muchas otras cosas (transporte de personas, mercancías, etc.).

Otros servicios que funcionan sobre Internet pero que no son la Web incluyen: el correo electrónico (protocolo SMTP), la transferencia de archivos (protocolo FTP), la mensajería instantánea, las videollamadas, los juegos en línea y muchos más.

Evolución de la Web

La Web ha evolucionado significativamente desde su creación. Se suele describir esta evolución en tres generaciones:

Web 1.0 — La Web estática (1991-2004 aproximadamente)

- Páginas web de solo lectura: el usuario consumía contenido pero no podía interactuar ni contribuir.
- Contenido creado por una minoría técnica (webmasters).
- Sitios web simples con texto, imágenes y enlaces.
- Diseño basado en tablas HTML.
- Nula o escasa interactividad.
- Ejemplo: las primeras páginas personales, directorios de enlaces.

Web 2.0 — La Web social e interactiva (2004-2020 aproximadamente)

- El usuario pasa de consumidor a creador de contenido.
- Aparición de redes sociales, blogs, wikis y plataformas colaborativas.
- Aplicaciones web ricas e interactivas (correo web, mapas interactivos, editores en línea).
- Tecnologías como AJAX, JavaScript avanzado y APIs permiten experiencias dinámicas.
- Diseño responsive que se adapta a múltiples dispositivos.
- Modelo de negocio basado en datos del usuario y publicidad.
- Ejemplo: redes sociales, plataformas de video, servicios de correo web.

Web 3.0 — La Web descentralizada e inteligente (2020 en adelante)

- Uso intensivo de inteligencia artificial y aprendizaje automático.
- Tendencia hacia la descentralización (tecnologías blockchain).

- Web semántica: los datos están estructurados de forma que las máquinas pueden interpretarlos.
- Mayor control del usuario sobre sus propios datos.
- Experiencias inmersivas (realidad virtual, realidad aumentada).
- Ejemplo: asistentes inteligentes, aplicaciones descentralizadas, generación de contenido con IA.

Es importante notar que estas categorías son más bien descriptivas y retrospectivas; no hay un momento exacto de transición de una a otra, y muchas características coexisten.

---

3. Páginas web, sitios web y tipos de sitios

Página web vs. sitio web

Aunque en el lenguaje cotidiano se usan indistintamente, técnicamente son conceptos diferentes:

- **Página web:** Un documento individual accesible a través de la Web. Tiene su propia URL y puede contener texto, imágenes, videos, formularios y otros elementos. Es análoga a una página de un libro.
- **Sitio web:** Un conjunto de páginas web relacionadas entre sí, organizadas bajo un mismo dominio y generalmente con una navegación común. Es análogo al libro completo.

Por ejemplo, un sitio web de una universidad puede tener cientos de páginas web: la página de inicio, la página de cada carrera, la página de contacto, etc. Todas forman parte del mismo sitio.

Tipos de sitios web

Los sitios web se pueden clasificar de diversas formas. Una clasificación técnica fundamental es:

Sitios web estáticos:

- El contenido está predefinido en archivos HTML/CSS y se muestra igual para todos los visitantes.
- No requieren procesamiento del lado del servidor más allá de enviar los archivos.
- Son rápidos, seguros y fáciles de alojar.
- Adecuados para sitios con contenido que cambia con poca frecuencia: portafolios, landing pages, sitios informativos, documentación.
- En esta asignatura trabajaremos con este tipo de sitios.

Sitios web dinámicos:

- El contenido se genera en tiempo real en el servidor a partir de datos almacenados en bases de datos.
- El contenido puede variar según el usuario, la hora, la ubicación u otros factores.
- Requieren un servidor con lenguajes de programación del lado del servidor y bases de datos.
- Adecuados para sitios con contenido que cambia frecuentemente o que es personalizado: blogs, tiendas en línea, redes sociales.

Aplicaciones web:

- Sitios web altamente interactivos que se comportan como aplicaciones de escritorio o móviles.
- Ofrecen funcionalidades complejas: edición de documentos, gestión de proyectos, comercio electrónico completo, herramientas de comunicación.
- Suelen combinar tecnologías del lado del cliente y del servidor con arquitecturas sofisticadas.
- Ejemplo: un editor de texto en línea, un sistema de gestión empresarial, una plataforma de correo web.

Otra clasificación común es por propósito: sitios corporativos/institucionales, tiendas en línea (e-commerce), blogs y medios, portafolios, wikis y documentación, foros y comunidades, plataformas de servicios, entre otros.

---

4. Arquitectura cliente-servidor

¿Qué es la arquitectura cliente-servidor?

La Web funciona bajo un modelo de arquitectura cliente-servidor, donde dos actores principales se comunican entre sí:

Cliente: Es el software que realiza solicitudes de recursos. En el contexto de la Web, el cliente es el navegador web (también llamado "user agent"). Cuando un usuario escribe una dirección en la barra del navegador o hace clic en un enlace, el navegador actúa como cliente: envía una solicitud al servidor y luego interpreta y muestra la respuesta recibida.

Servidor: Es el software (y por extensión, la máquina que lo ejecuta) que almacena los recursos y responde a las solicitudes de los clientes. Un servidor web recibe solicitudes HTTP, busca el recurso solicitado y lo envía de vuelta al cliente. Si el recurso no existe, responde con un mensaje de error.

Flujo básico de comunicación

El proceso de cargar una página web sigue estos pasos simplificados:

1. El usuario escribe una URL en el navegador o hace clic en un enlace.
2. El navegador consulta al DNS para obtener la dirección IP del servidor.
3. El navegador establece una conexión TCP con el servidor.
4. El navegador envía una solicitud HTTP al servidor pidiendo el recurso.
5. El servidor procesa la solicitud y envía una respuesta HTTP con el recurso (por ejemplo, un archivo HTML).
6. El navegador recibe la respuesta y comienza a interpretar el HTML.
7. Si el HTML referencia otros recursos (hojas de estilo CSS, imágenes, scripts), el navegador envía solicitudes adicionales para obtenerlos.
8. El navegador renderiza la página completa y la muestra al usuario.

Este proceso ocurre en fracciones de segundo y puede involucrar decenas o cientos de solicitudes para una sola página.

Otros actores en la comunicación

En la práctica, entre el cliente y el servidor pueden intervenir otros componentes:

- **Proxy:** Un intermediario que puede filtrar, cachear o modificar las solicitudes y respuestas.
- **CDN (Content Delivery Network):** Una red de servidores distribuidos geográficamente que almacenan copias de los recursos para entregarlos más rápido al usuario desde la ubicación más cercana.
- **Balanceador de carga:** Distribuye las solicitudes entre múltiples servidores para manejar grandes volúmenes de tráfico.
- **Firewall:** Filtra el tráfico de red por motivos de seguridad.

---

5. URLs y DNS

Anatomía de una URL

Una URL (Uniform Resource Locator) es la dirección que identifica de forma única un recurso en la Web. Tiene la siguiente estructura:

```
https://www.ejemplo.com:443/ruta/pagina.html?clave=valor&otra=dato#seccion
```

Cada parte cumple una función específica:

- **Esquema (protocolo):** `https://` — Indica el protocolo de comunicación. Los más comunes en la Web son `http://` y `https://` (la versión segura con cifrado).
- **Subdominio:** `www.` — Es un prefijo opcional del dominio. Otros subdominios comunes son `blog.`, `mail.`, `api.`, etc.
- **Dominio:** `ejemplo.com` — El nombre legible que identifica al servidor. Está compuesto por un nombre (`ejemplo`) y un dominio de nivel superior o TLD (`com`).
- **Puerto:** `:443` — El número de puerto del servidor. Generalmente se omite porque los protocolos tienen puertos por defecto: 80 para HTTP y 443 para HTTPS.
- **Ruta (path):** `/ruta/pagina.html` — La ubicación del recurso dentro del servidor. Funciona como una estructura de carpetas y archivos.
- **Parámetros de consulta (query string):** `?clave=valor&otra=dato` — Datos adicionales enviados al servidor, separados por `&`. Se usan para filtros, búsquedas, configuraciones, etc.
- **Fragmento (hash/ancla):** `#seccion` — Apunta a una sección específica dentro del documento. El navegador se desplaza automáticamente a esa posición. Este dato no se envía al servidor.

Sistema de Nombres de Dominio (DNS)

Los humanos recordamos nombres como `ejemplo.com`, pero las computadoras necesitan direcciones IP numéricas (como `93.184.216.34`) para comunicarse. El DNS (Domain Name System) es el sistema que traduce los nombres de dominio en direcciones IP. Se lo suele llamar "la guía telefónica de Internet".

El proceso de resolución DNS funciona así (simplificado):

1. El usuario escribe `www.ejemplo.com` en el navegador.
2. El navegador consulta su caché local: ¿ya conozco la IP de este dominio? Si sí, la usa directamente.
3. Si no está en caché, consulta al servidor DNS del proveedor de Internet (ISP).
4. Si el ISP no la tiene, se inicia una consulta recursiva: el servidor DNS pregunta a los servidores raíz, luego a los servidores del TLD (`.com`), y finalmente al servidor autoritativo del dominio `ejemplo.com`.
5. El servidor autoritativo responde con la dirección IP del servidor.
6. La respuesta viaja de vuelta al navegador, que ahora puede conectarse al servidor correcto.

Todo este proceso suele completarse en milisegundos.

Dominios de nivel superior (TLD)

El TLD es la última parte del dominio. Existen varios tipos:

- **Genéricos (gTLD):** `.com`, `.org`, `.net`, `.info`, `.edu`
- **De código de país (ccTLD):** `.ar` (Argentina), `.br` (Brasil), `.uk` (Reino Unido), `.de` (Alemania)
- **Nuevos gTLD:** `.tech`, `.dev`, `.app`, `.shop`, `.blog` y cientos más.

---

6. Protocolo HTTP/HTTPS

¿Qué es HTTP?

HTTP (HyperText Transfer Protocol) es el protocolo de la capa de aplicación que define cómo se comunican los clientes y los servidores en la Web. Es un protocolo basado en texto, sin estado (stateless) y que sigue un modelo de solicitud-respuesta.

Basado en texto significa que los mensajes HTTP son legibles por humanos (a diferencia de protocolos binarios).

Sin estado (stateless) significa que cada solicitud es independiente: el servidor no recuerda solicitudes anteriores. Cada solicitud debe contener toda la información necesaria para ser procesada. Para mantener el "estado" (por ejemplo, que un usuario esté logueado), se utilizan mecanismos adicionales como cookies y tokens.

HTTPS: HTTP seguro

HTTPS (HTTP Secure) es la versión segura de HTTP. Utiliza cifrado TLS (Transport Layer Security) para proteger la comunicación entre el cliente y el servidor. Esto garantiza:

- **Confidencialidad:** Nadie puede leer los datos en tránsito.
- **Integridad:** Los datos no pueden ser modificados durante la transmisión.
- **Autenticación:** El cliente puede verificar que está hablando con el servidor correcto.

Hoy en día, HTTPS es el estándar esperado para todos los sitios web. Los navegadores marcan como "No seguro" a los sitios que usan HTTP plano.

Métodos HTTP

Los métodos HTTP (también llamados "verbos") indican la acción que el cliente quiere realizar sobre un recurso. Los más comunes son:

GET: Solicita un recurso. Es el método más frecuente; se usa cada vez que el navegador carga una página, una imagen, una hoja de estilos, etc. No modifica datos en el servidor.

POST: Envía datos al servidor para crear o procesar un recurso. Se usa típicamente al enviar formularios, subir archivos o hacer una compra.

PUT: Reemplaza completamente un recurso existente en el servidor con los datos enviados.

PATCH: Modifica parcialmente un recurso existente.

DELETE: Elimina un recurso del servidor.

HEAD: Igual que GET, pero el servidor solo devuelve las cabeceras (sin el cuerpo). Útil para verificar si un recurso existe o consultar sus metadatos.

OPTIONS: Consulta qué métodos y opciones están disponibles para un recurso.

En el desarrollo de sitios web estáticos, los métodos que más veremos son GET (para cargar páginas y recursos) y POST (para enviar formularios).

Códigos de estado HTTP

Cuando el servidor responde a una solicitud, incluye un código de estado numérico de tres dígitos que indica el resultado. Se agrupan por familias:

1xx — Informativos: La solicitud fue recibida y se está procesando. (Poco frecuentes en el uso cotidiano.)

2xx — Éxito: La solicitud fue recibida, comprendida y procesada correctamente.

- `200 OK` — La solicitud fue exitosa. El recurso se devuelve en el cuerpo de la respuesta.
- `201 Created` — Se creó un nuevo recurso exitosamente (típico en respuestas a POST).
- `204 No Content` — La solicitud fue exitosa pero no hay contenido que devolver.

3xx — Redirección: El recurso se movió y el cliente debe ir a otra dirección.

- `301 Moved Permanently` — El recurso se movió permanentemente a otra URL.
- `302 Found` — El recurso se movió temporalmente.

- `304 Not Modified` — El recurso no cambió desde la última vez que se solicitó; el cliente puede usar su copia en caché.

4xx — Error del cliente: La solicitud tiene algún problema.

- `400 Bad Request` — La solicitud tiene errores de sintaxis o parámetros inválidos.
- `401 Unauthorized` — Se requiere autenticación para acceder al recurso.
- `403 Forbidden` — El servidor entiende la solicitud pero se niega a cumplirla (el cliente no tiene permisos).
- `404 Not Found` — El recurso solicitado no existe en el servidor. Es probablemente el código de error más conocido.
- `405 Method Not Allowed` — El método HTTP utilizado no está permitido para ese recurso.

5xx — Error del servidor: El servidor falló al procesar una solicitud aparentemente válida.

- `500 Internal Server Error` — Error genérico del servidor.
- `502 Bad Gateway` — Un servidor intermediario recibió una respuesta inválida del servidor de origen.
- `503 Service Unavailable` — El servidor no puede manejar la solicitud temporalmente (sobrecarga, mantenimiento).

Cabeceras HTTP

Las cabeceras (headers) son metadatos que acompañan tanto a las solicitudes como a las respuestas HTTP. Proporcionan información adicional sobre la comunicación. Algunas cabeceras frecuentes:

En la solicitud (del cliente al servidor):

- `Host` — El nombre del dominio al que se dirige la solicitud. Ejemplo: `Host: www.ejemplo.com`
- `User-Agent` — Información sobre el navegador y sistema operativo del cliente.
- `Accept` — Tipos de contenido que el cliente puede procesar. Ejemplo: `Accept: text/html, application/json`
- `Accept-Language` — Idiomas preferidos por el usuario. Ejemplo: `Accept-Language: es-AR, es, en`
- `Cookie` — Datos de sesión almacenados previamente.

En la respuesta (del servidor al cliente):

- `Content-Type` — El tipo de contenido que se envía. Ejemplo: `Content-Type: text/html; charset=UTF-8`
- `Content-Length` — El tamaño del cuerpo de la respuesta en bytes.
- `Cache-Control` — Instrucciones sobre cómo cachear el recurso.
- `Set-Cookie` — Establece una cookie en el navegador del cliente.
- `Location` — URL de redirección (usada con códigos 3xx).

Estructura de una solicitud HTTP

Una solicitud HTTP tiene la siguiente estructura:

```
GET /pagina.html HTTP/1.1
Host: www.ejemplo.com
User-Agent: Mozilla/5.0
Accept: text/html
Accept-Language: es-AR
```

La primera línea contiene el método, la ruta del recurso y la versión del protocolo. Las líneas siguientes son las cabeceras. Si el método lo requiere (como POST), después de las cabeceras y una línea en blanco viene el cuerpo con los datos.

Estructura de una respuesta HTTP

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 3542

<!DOCTYPE html>
<html>
  ...contenido de la página...
</html>
```

La primera línea contiene la versión del protocolo, el código de estado y su descripción. Luego vienen las cabeceras, una línea en blanco, y finalmente el cuerpo de la respuesta (en este caso, el documento HTML).

---

7. Herramientas de desarrollo del navegador

¿Qué son las DevTools?

Las herramientas de desarrollo del navegador (comúnmente llamadas DevTools) son un conjunto de utilidades integradas en los navegadores modernos que permiten inspeccionar, depurar y analizar el comportamiento de las páginas web. Son una herramienta esencial para todo desarrollador web.

Para abrirlas se puede usar el atajo de teclado `F12` o `Ctrl+Shift+I` (en Windows/Linux) / `Cmd+Option+I` (en macOS), o hacer clic derecho en cualquier elemento de la página y seleccionar "Inspeccionar".

Pestaña Elementos (Elements)

La pestaña Elementos permite ver y manipular el HTML y CSS de la página en tiempo real:

- **Árbol del DOM:** Muestra la estructura HTML completa del documento como un árbol jerárquico. Se puede expandir y contraer cada elemento.
- **Inspección de elementos:** Al seleccionar un elemento en el árbol, se resalta en la página. También se puede hacer clic derecho en cualquier elemento visible de la página y elegir "Inspeccionar" para ir directamente a ese nodo en el árbol.
- **Edición en vivo:** Se pueden modificar atributos, contenido de texto y etiquetas directamente en el árbol, y los cambios se reflejan inmediatamente en la página. Estos cambios son temporales: se pierden al recargar.
- **Panel de estilos:** A la derecha del árbol se muestran todos los estilos CSS aplicados al elemento seleccionado, incluyendo de qué regla provienen, su especificidad y qué propiedades están siendo sobrescritas.
- **Modelo de caja (box model):** Una representación visual del modelo de caja del elemento seleccionado, mostrando las dimensiones del contenido, padding, border y margin.

Pestaña Red (Network)

La pestaña Red muestra todas las solicitudes HTTP que el navegador realiza al cargar una página:

- **Lista de solicitudes:** Cada fila es una solicitud individual (el documento HTML, cada archivo CSS, cada imagen, cada script, cada fuente, etc.).
- **Información por solicitud:** Al hacer clic en una solicitud se puede ver: el método HTTP utilizado, la URL completa, los códigos de estado, las cabeceras de solicitud y respuesta, el cuerpo de la respuesta, y el tiempo que tardó.
- **Filtros:** Se pueden filtrar las solicitudes por tipo (documentos, hojas de estilo, imágenes, scripts, fuentes, etc.).

- **Línea de tiempo:** Muestra gráficamente cuánto tardó cada solicitud y en qué orden se ejecutaron.
- **Resumen:** Al pie se muestra el total de solicitudes, el peso total de los datos transferidos y el tiempo de carga.

Para ver las solicitudes de red de una página, es importante abrir la pestaña Red **antes** de cargar la página (o recargarla con la pestaña abierta).

Otras pestañas útiles

- **Consola:** Muestra mensajes de error, advertencias y permite ejecutar código JavaScript.
- **Fuentes (Sources):** Permite ver el código fuente de todos los archivos cargados.
- **Aplicación (Application):** Muestra cookies, almacenamiento local y otros datos del sitio.
- **Rendimiento (Performance):** Herramientas de análisis de performance para detectar cuellos de botella.

A lo largo de la asignatura iremos descubriendo más funcionalidades de las DevTools a medida que las necesitemos.

---

8. Introducción a HTML5

¿Qué es HTML?

HTML (HyperText Markup Language) es el lenguaje de marcado estándar para crear páginas web. No es un lenguaje de programación (no tiene lógica, variables, condiciones ni bucles); es un lenguaje de marcado que define la estructura y el contenido de un documento web mediante etiquetas.

"HyperText" hace referencia a que los documentos HTML pueden contener hipervínculos (enlaces) que conectan a otros documentos, formando la red de documentos interconectados que es la Web. "Markup" significa que el contenido se anota con etiquetas que le dan estructura y significado.

Breve historia de HTML

- **1991 — HTML original:** Tim Berners-Lee publicó la primera versión con 18 etiquetas.
- **1995 — HTML 2.0:** Primera especificación formal como estándar.
- **1997 — HTML 3.2 y HTML 4.0:** Agregaron tablas, scripts y hojas de estilo.
- **1999 — HTML 4.01:** Versión refinada que fue el estándar durante más de una década.
- **2000 — XHTML:** Intento de reformular HTML con las reglas estrictas de XML. Tuvo adopción limitada.
- **2014 — HTML5:** La versión actual, desarrollada por el WHATWG y el W3C. Introdujo nuevos elementos semánticos, soporte nativo para multimedia, APIs para aplicaciones web y mucho más.

HTML5 no es solo una actualización del lenguaje de marcado: es un conjunto de tecnologías que incluye el lenguaje HTML propiamente dicho, nuevas APIs de JavaScript y especificaciones asociadas. Cuando hablamos de "HTML5" en esta asignatura nos referimos a la versión actual del lenguaje HTML.

Etiquetas y elementos

HTML utiliza **etiquetas** para marcar el contenido. La mayoría de las etiquetas tienen una etiqueta de apertura y una de cierre:

```
<p>Este es un párrafo.</p>
```

- `<p>` es la etiqueta de apertura.
- `</p>` es la etiqueta de cierre (tiene la barra `/`).
- `Este es un párrafo.` es el contenido.
- Todo junto (etiqueta de apertura + contenido + etiqueta de cierre) es un **elemento**.

Algunas etiquetas son **vacías** (void elements): no tienen contenido ni etiqueta de cierre:

```
<br>
<img src="foto.jpg" alt="Descripción de la foto">
<hr>
<input type="text">
```

Atributos

Las etiquetas pueden tener **atributos** que proporcionan información adicional sobre el elemento. Se escriben en la etiqueta de apertura como pares `nombre="valor"` :

```
<a href="https://ejemplo.com" target="_blank">Visitar ejemplo</a>
<img src="foto.jpg" alt="Descripción" width="300" height="200">
```

Algunos atributos comunes a todos los elementos:

- `id` — Identificador único del elemento dentro del documento.
- `class` — Una o más clases CSS asignadas al elemento (pueden repetirse en varios elementos).
- `style` — Estilos CSS aplicados directamente al elemento (inline).
- `title` — Texto informativo que aparece al pasar el cursor por encima.
- `lang` — Idioma del contenido del elemento.

Anidamiento

Los elementos HTML se pueden anidar (colocar unos dentro de otros), formando una estructura jerárquica en forma de árbol:

```
<article>
  <h2>Título del artículo</h2>
  <p>Este párrafo contiene <strong>texto en negrita</strong> y un <a href="#">enlace</a>.
</p>
</article>
```

Es fundamental que los elementos se cierren en el orden correcto (el último en abrirse es el primero en cerrarse):

```
<!-- Correcto -->
<p>Texto <strong>en negrita</strong></p>

<!-- Incorrecto -->
<p>Texto <strong>en negrita</p></strong>
```

---

9. Estructura de un documento HTML5

Estructura mínima

Todo documento HTML5 tiene una estructura base obligatoria:

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Título de la página</title>
</head>
<body>
  <!-- El contenido visible de la página va aquí -->
</body>
</html>
```

Cada parte cumple una función:

`<!DOCTYPE html>` — La declaración de tipo de documento. Le indica al navegador que el documento usa HTML5. Debe ser la primera línea del archivo. No es una etiqueta HTML, sino una instrucción para el navegador.

`<html lang="es">` — El elemento raíz que contiene todo el documento. El atributo `lang` indica el idioma principal del contenido (importante para accesibilidad y SEO). Para español de Argentina se puede usar `lang="es-AR"`.

`<head>` — La cabecera del documento. Contiene metadatos, enlaces a recursos y configuraciones que no se muestran directamente en la página pero son esenciales para su funcionamiento.

`<meta charset="UTF-8">` — Define la codificación de caracteres del documento. UTF-8 es el estándar universal que soporta prácticamente todos los caracteres de todos los idiomas, incluyendo acentos y la ñ.

`<meta name="viewport" ...>` — Configura cómo se muestra la página en dispositivos móviles. Sin esta etiqueta, los móviles podrían mostrar la página como si fuera de escritorio, muy reducida. `width=device-width` ajusta el ancho al del dispositivo, e `initial-scale=1.0` establece el zoom inicial en 100%.

`<title>` — El título del documento. Aparece en la pestaña del navegador, en los resultados de los motores de búsqueda y cuando se guarda la página como favorito. Cada página debe tener un título único y descriptivo.

`<body>` — El cuerpo del documento. Contiene todo el contenido visible de la página: texto, imágenes, enlaces, formularios, etc.

`<!-- comentario -->` — Los comentarios en HTML se escriben entre `<!--` y `-->`. No se muestran en la página pero son visibles en el código fuente. Son útiles para documentar el código.

Elementos comunes dentro de `<head>`

Además de los ya mencionados, el `<head>` suele contener:

```html
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Breve descripción de la página para buscadores">
  <meta name="author" content="Nombre del autor">
  <title>Título de la página</title>
  <link rel="stylesheet" href="estilos.css">
```

```
  <link rel="icon" href="favicon.ico">
</head>
```

- `<meta name="description">` — Descripción de la página que los motores de búsqueda pueden mostrar en sus resultados.
- `<meta name="author">` — Autor del documento.
- `<link rel="stylesheet">` — Enlace a una hoja de estilos CSS externa.
- `<link rel="icon">` — El favicon, el pequeño ícono que aparece en la pestaña del navegador.

---

10. Etiquetas básicas de HTML

Encabezados

HTML define seis niveles de encabezados, de `<h1>` (el más importante) a `<h6>` (el menos importante):

```
<h1>Encabezado de nivel 1</h1>
<h2>Encabezado de nivel 2</h2>
<h3>Encabezado de nivel 3</h3>
<h4>Encabezado de nivel 4</h4>
<h5>Encabezado de nivel 5</h5>
<h6>Encabezado de nivel 6</h6>
```

Reglas importantes sobre los encabezados:

- Cada página debe tener un solo `<h1>`, que describe el tema principal de la página.
- Los encabezados deben seguir un orden jerárquico lógico: no saltar de `<h1>` a `<h3>` sin un `<h2>` intermedio.
- Los encabezados no se eligen por su tamaño visual (eso se controla con CSS) sino por su nivel de importancia en la jerarquía del contenido.

Párrafos y texto

```
<p>Este es un párrafo de texto. Los párrafos son bloques de texto separados
por un espacio vertical. HTML ignora los saltos de línea y espacios múltiples
en el código fuente.</p>

<p>Este es otro párrafo separado.</p>
```

Para formatear texto dentro de un párrafo:

```
<p>
  Texto <strong>en negrita (importancia fuerte)</strong> y
  texto <em>en cursiva (énfasis)</em>.
  También existe <b>negrita visual</b> y <i>cursiva visual</i>
  (sin significado semántico adicional).
</p>

<p>
  Texto <mark>resaltado</mark>,
  texto <small>pequeño</small>,
```

```
  texto <del>tachado (eliminado)</del> y
  texto <ins>subrayado (insertado)</ins>.
</p>
```

La diferencia entre `<strong>` y `<b>` (y entre `<em>` e `<i>`) es semántica: `<strong>` indica que el texto tiene importancia fuerte, mientras que `<b>` solo aplica un estilo visual sin significado adicional. Se recomienda usar las etiquetas semánticas (`<strong>` y `<em>`) cuando corresponda.

Saltos de línea y líneas horizontales

```
<p>Primera línea<br>Segunda línea del mismo párrafo</p>

<hr>
<!-- hr crea una línea horizontal, útil como separador temático -->
```

`<br>` fuerza un salto de línea dentro de un bloque. No debe usarse para crear espaciado entre elementos (para eso se usa CSS).

Enlaces

Los enlaces o hipervínculos son la esencia de la Web:

```
<!-- Enlace a otra página -->
<a href="https://ejemplo.com">Visitar ejemplo</a>

<!-- Enlace que abre en nueva pestaña -->
<a href="https://ejemplo.com" target="_blank" rel="noopener noreferrer">Abrir en nueva
pestaña</a>

<!-- Enlace a otra página del mismo sitio -->
<a href="contacto.html">Ir a contacto</a>

<!-- Enlace a una sección de la misma página (ancla) -->
<a href="#seccion-2">Ir a la sección 2</a>

<!-- Enlace de correo electrónico -->
<a href="mailto:correo@ejemplo.com">Enviar correo</a>

<!-- Enlace de teléfono -->
<a href="tel:+541234567890">Llamar</a>
```

El atributo `href` (hypertext reference) contiene la URL de destino. El atributo `target="_blank"` abre el enlace en una nueva pestaña, y `rel="noopener noreferrer"` es una buena práctica de seguridad cuando se usa `target="_blank"` .

Imágenes

```
<img src="foto.jpg" alt="Descripción de la imagen" width="600" height="400">
```

- `src` — La ruta o URL de la imagen.
- `alt` — Texto alternativo que describe la imagen. Es obligatorio y fundamental para la accesibilidad: es lo que leen los lectores de pantalla y lo que se muestra si la imagen no carga.
- `width` y `height` — Dimensiones de la imagen. Especificarlas ayuda al navegador a reservar el espacio antes de que la imagen se cargue, evitando "saltos" en el diseño.

Listas

HTML ofrece tres tipos de listas:

```
<!-- Lista desordenada (viñetas) -->
<ul>
  <li>Primer elemento</li>
  <li>Segundo elemento</li>
  <li>Tercer elemento</li>
</ul>

<!-- Lista ordenada (numerada) -->
<ol>
  <li>Primer paso</li>
  <li>Segundo paso</li>
  <li>Tercer paso</li>
</ol>

<!-- Lista de definiciones -->
<dl>
  <dt>HTML</dt>
  <dd>Lenguaje de marcado para estructurar contenido web.</dd>
  <dt>CSS</dt>
  <dd>Lenguaje de hojas de estilo para diseñar la presentación visual.</dd>
</dl>
```

Las listas pueden anidarse (listas dentro de listas) para crear jerarquías.

Contenedores genéricos

Cuando ningún elemento semántico es apropiado, se pueden usar contenedores genéricos:

```
<!-- Contenedor de bloque (ocupa todo el ancho disponible) -->
<div>
  <p>Contenido agrupado en un bloque.</p>
</div>

<!-- Contenedor en línea (ocupa solo el espacio de su contenido) -->
<p>Este texto tiene una <span>palabra resaltada</span>.</p>
```

`<div>` y `<span>` no tienen significado semántico propio. Se usan como "ganchos" para aplicar estilos CSS o agrupar contenido cuando no existe un elemento semántico más adecuado. Sin embargo, antes de usar un `<div>`, siempre hay que preguntarse si existe un elemento semántico que describa mejor el contenido (como `<section>`, `<article>`, `<nav>`, etc.).

---

11. Elementos semánticos

¿Qué es la semántica en HTML?

La semántica se refiere al significado de las etiquetas. Un elemento semántico describe claramente su contenido tanto al navegador como al desarrollador. Por ejemplo, `<nav>` indica que su contenido es una sección de navegación, mientras que `<div>` no dice nada sobre lo que contiene.

¿Por qué importa la semántica?

La semántica es importante por varias razones:

- **Accesibilidad:** Los lectores de pantalla y otras tecnologías asistivas usan los elementos semánticos para navegar y entender la estructura de la página. Una persona con discapacidad visual puede pedirle a su lector de pantalla "ir al contenido principal" si existe un `<main>`, o "listar los encabezados" para tener un resumen rápido del contenido.
- **SEO:** Los motores de búsqueda usan la estructura semántica para entender de qué trata cada parte de la página y determinar su relevancia.
- **Mantenibilidad:** El código es más fácil de leer y mantener cuando los elementos describen su función. `<nav>` es mucho más claro que `<div class="navegacion">`.
- **Consistencia:** Proporcionan un vocabulario compartido entre desarrolladores.

Elementos semánticos estructurales de HTML5

HTML5 introdujo un conjunto de elementos semánticos que describen la estructura de la página:

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sitio de ejemplo</title>
</head>
<body>

  <header>
    <h1>Nombre del sitio</h1>
    <nav>
      <ul>
        <li><a href="/">Inicio</a></li>
        <li><a href="/servicios">Servicios</a></li>
        <li><a href="/contacto">Contacto</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <section>
      <h2>Nuestros servicios</h2>
      <article>
        <h3>Diseño web</h3>
        <p>Creamos sitios web a medida...</p>
```

```
      </article>
      <article>
        <h3>Desarrollo de aplicaciones</h3>
        <p>Desarrollamos aplicaciones web...</p>
      </article>
    </section>

    <aside>
      <h2>Artículos relacionados</h2>
      <ul>
        <li><a href="#">Tendencias de diseño 2026</a></li>
        <li><a href="#">Introducción a la accesibilidad</a></li>
      </ul>
    </aside>
  </main>

  <footer>
    <p>&copy; 2026 — Nombre del sitio. Todos los derechos reservados.</p>
  </footer>

</body>
</html>
```

Cada elemento cumple un rol específico:

`<header>` — Encabezado introductorio de la página o de una sección. Generalmente contiene el logotipo, el título del sitio y la navegación principal. Puede haber varios `<header>` en una página (uno por sección), pero típicamente hay uno principal al inicio del `<body>`.

`<nav>` — Sección de navegación. Contiene los enlaces principales del sitio o de una sección. No todos los grupos de enlaces necesitan un `<nav>`; se reserva para bloques de navegación principales (menú del sitio, tabla de contenidos, paginación).

`<main>` — El contenido principal y único de la página. Solo debe haber un `<main>` por página, y no debe estar anidado dentro de `<header>`, `<footer>`, `<nav>` ni `<aside>`. Representa el contenido que es específico de esta página y no se repite en otras (a diferencia del encabezado y el pie, que suelen ser iguales en todas las páginas del sitio).

`<section>` — Una sección temática genérica del documento. Agrupa contenido relacionado temáticamente y normalmente tiene su propio encabezado (`<h2>`, `<h3>`, etc.). Se usa cuando una porción de contenido tiene un tema específico pero no se ajusta a `<article>`, `<nav>` u otro elemento más específico.

`<article>` — Un contenido autónomo e independiente que tendría sentido por sí solo fuera del contexto de la página. Ejemplos: una entrada de blog, un comentario de usuario, una noticia, una ficha de producto, un widget interactivo. La prueba mental: ¿este contenido tendría sentido si se compartiera por separado (por ejemplo, en un feed RSS)? Si sí, es un `<article>`.

`<aside>` — Contenido tangencialmente relacionado con el contenido principal. Puede ser una barra lateral, una caja de información complementaria, publicidad, enlaces relacionados o una biografía del autor. No es "menos importante" sino "complementario".

`<footer>` — Pie de la página o de una sección. Generalmente contiene información sobre el autor, derechos de autor, enlaces legales, datos de contacto. Al igual que `<header>`, puede haber varios `<footer>` en una página.

Otros elementos semánticos útiles

`<figure>` y `<figcaption>` — Para imágenes, diagramas, ilustraciones o fragmentos de código con una leyenda:

```
<figure>
  <img src="grafico.png" alt="Gráfico de barras mostrando las ventas por trimestre">
  <figcaption>Figura 1: Ventas trimestrales del año 2025.</figcaption>
</figure>
```

`<time>` — Para representar fechas y horas de forma legible por máquinas:

```
<p>Publicado el <time datetime="2026-03-15">15 de marzo de 2026</time>.</p>
```

`<address>` — Para información de contacto del autor o del propietario del documento:

```
<address>
  Contacto: <a href="mailto:info@ejemplo.com">info@ejemplo.com</a>
</address>
```

Errores comunes con la semántica

- **Usar `<div>` para todo:** Si hay un elemento semántico que describe el contenido, debe preferirse sobre `<div>`.
- **Usar `<section>` como reemplazo de `<div>`:** `<section>` implica agrupación temática y debería tener un encabezado. Si solo se necesita un contenedor para estilos, `<div>` es más apropiado.
- **Confundir `<section>` y `<article>`:** El criterio clave es la autonomía. Un `<article>` es independiente y autocontenido; una `<section>` es una agrupación temática dentro de un contexto mayor.
- **Múltiples `<main>`:** Solo debe haber un `<main>` por página.
- **Usar encabezados (`<h1>` - `<h6>`) por su tamaño visual:** El nivel del encabezado debe reflejar la jerarquía del contenido, no su apariencia.

---

12. Validación según estándares del W3C

¿Qué es el W3C?

El W3C (World Wide Web Consortium) es la organización internacional que desarrolla y mantiene los estándares de la Web, incluyendo las especificaciones de HTML y CSS. Fue fundado en 1994 por Tim Berners-Lee con el objetivo de guiar el desarrollo de la Web hacia su máximo potencial.

¿Qué significa validar un documento HTML?

Validar un documento HTML significa verificar que su código cumple con las reglas de la especificación del lenguaje. Un documento válido:

- Tiene la estructura correcta (DOCTYPE, html, head, body).
- Usa las etiquetas correctamente (etiquetas existentes, correctamente anidadas, correctamente cerradas).
- Tiene los atributos obligatorios (por ejemplo, `alt` en las imágenes).
- No contiene errores de sintaxis.

¿Por qué validar?

- **Compatibilidad:** Un documento válido tiene mayor probabilidad de mostrarse correctamente en todos los navegadores.
- **Accesibilidad:** Muchos errores de validación también son problemas de accesibilidad.
- **Mantenibilidad:** El código válido es más predecible y fácil de mantener.
- **Depuración:** Validar ayuda a detectar errores que podrían causar comportamientos inesperados.
- **SEO:** Los motores de búsqueda pueden tener dificultades para interpretar correctamente documentos con errores graves de marcado.

El validador del W3C

El W3C ofrece un validador gratuito en línea donde se puede verificar el código HTML. Se puede usar de tres formas:

- **Por URL:** Se ingresa la dirección de una página publicada y el validador la analiza.
- **Por carga de archivo:** Se sube un archivo HTML local.
- **Por entrada directa:** Se pega el código HTML directamente en un campo de texto.

El validador analiza el código y muestra:

- **Errores (errors):** Problemas que violan la especificación y deben corregirse. Ejemplo: una etiqueta `<img>` sin atributo `alt`, un elemento sin su etiqueta de cierre, una etiqueta inexistente.
- **Advertencias (warnings):** Problemas menores o recomendaciones que no violan la especificación pero son buenas prácticas. Ejemplo: falta el atributo `lang` en el elemento `<html>`.

Errores de validación más comunes

- Elementos no cerrados o cerrados en orden incorrecto.
- Atributo `alt` faltante en etiquetas `<img>`.
- Elementos anidados incorrectamente (por ejemplo, un `<div>` dentro de un `<span>`).
- Atributos duplicados en un mismo elemento.
- Uso de etiquetas o atributos obsoletos.
- Falta de la declaración `<!DOCTYPE html>`.
- Caracteres especiales sin codificar (como `&` en lugar de `&` dentro de URLs en atributos).

Ejemplo de validación

Consideremos el siguiente documento con errores intencionales:

```
<!DOCTYPE html>
<html>
<head>
  <title>Mi página</title>
</head>
<body>
  <h1>Bienvenidos
  <p>Este es un párrafo con <strong>texto en negrita</p></strong>
  <img src="foto.jpg">
</body>
</html>
```

El validador detectaría al menos estos errores:

1. Falta el atributo `lang` en `<html>` (advertencia).
2. Falta `<meta charset="UTF-8">` (advertencia).
3. El `<h1>` no está cerrado.

4. `<strong>` y `<p>` están cerrados en orden incorrecto.
5. La etiqueta `<img>` no tiene atributo `alt`.

La versión corregida sería:

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mi página</title>
</head>
<body>
  <h1>Bienvenidos</h1>
  <p>Este es un párrafo con <strong>texto en negrita</strong></p>
  <img src="foto.jpg" alt="Descripción de la foto">
</body>
</html>
```

Validar el código debe ser un hábito durante todo el desarrollo, no algo que se haga solo al final. A medida que avancemos en la asignatura, se espera que todos los trabajos prácticos entreguen código HTML válido.

Resumen de la unidad

En esta unidad hemos recorrido los cimientos sobre los que se construye toda la Web:

- **Internet** es la red global de redes; **la Web** es un servicio que funciona sobre ella.
- La Web funciona con una **arquitectura cliente-servidor** donde el navegador solicita recursos y el servidor los entrega.
- Las **URLs** identifican cada recurso en la Web, y el **DNS** traduce nombres de dominio en direcciones IP.
- **HTTP** es el protocolo de comunicación de la Web, basado en solicitudes y respuestas con métodos (GET, POST, etc.) y códigos de estado (200, 404, 500, etc.).
- Las **herramientas de desarrollo del navegador** permiten inspeccionar la estructura HTML, los estilos CSS y las comunicaciones HTTP de cualquier página.
- **HTML5** es el lenguaje de marcado que define la estructura y el contenido de las páginas web, utilizando etiquetas con significado semántico.
- Los **elementos semánticos** (`header`, `nav`, `main`, `section`, `article`, `aside`, `footer`) describen la función de cada parte de la página, mejorando la accesibilidad, el SEO y la mantenibilidad.
- **Validar** el código HTML según los estándares del W3C es una práctica esencial para garantizar la calidad y la compatibilidad del sitio.

Lecturas recomendadas

- Documentación abierta de referencia para HTML (MDN Web Docs u equivalente): secciones sobre estructura HTML, elementos semánticos y referencia de etiquetas.
- Duckett, J. (2011). *HTML and CSS: Design and Build Websites*. Wiley. Capítulos 1 a 7.